

Encoding Network PlusCal into the Join Calculus

AFADL '26

Ghilain BERGERON

Université de Lorraine,
CNRS, Inria, LORIA

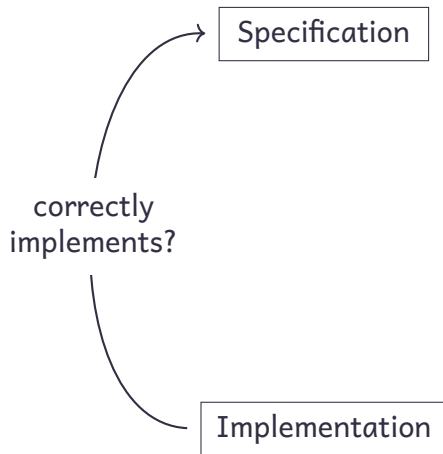
June 1, 2026

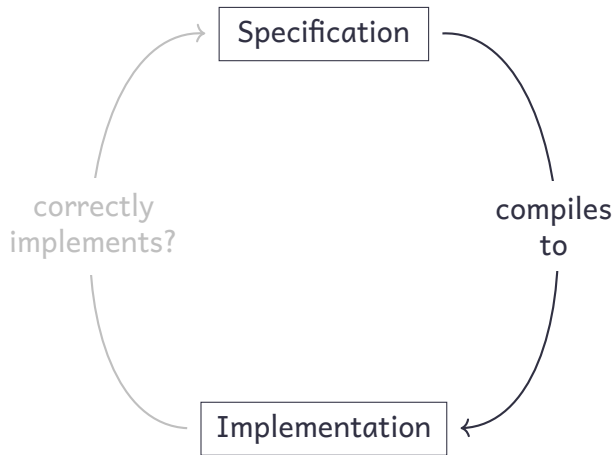


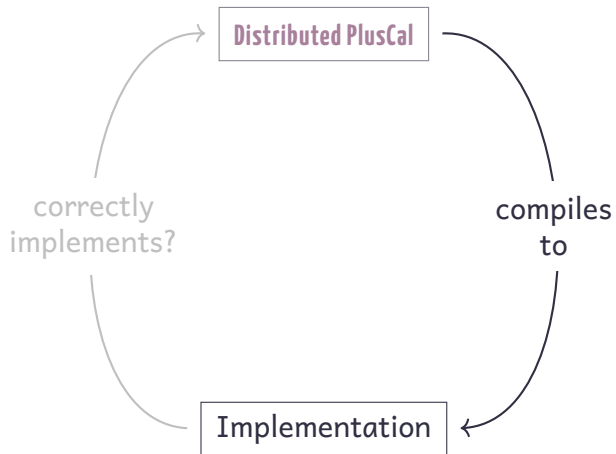
Implementation

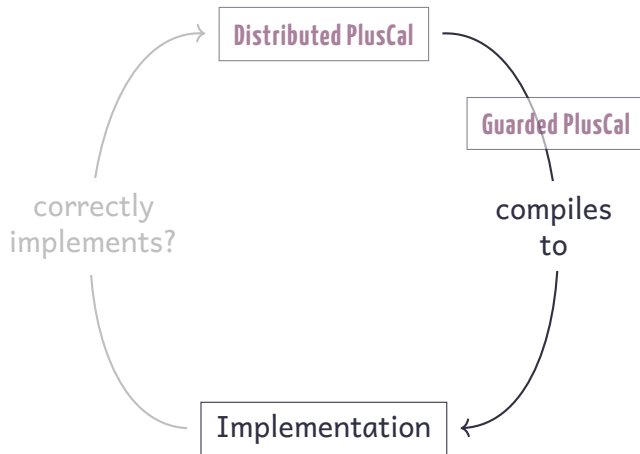
Specification

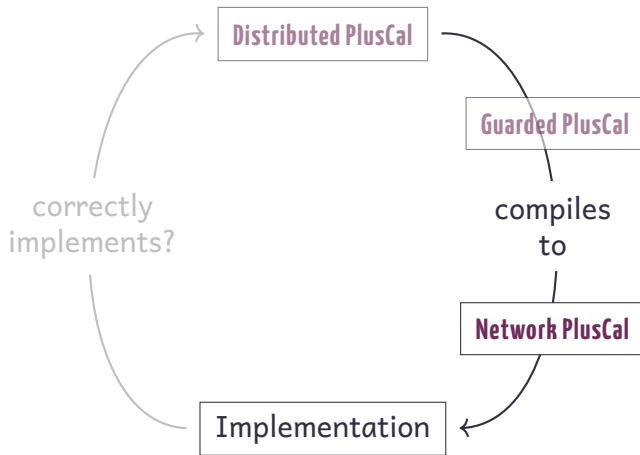
Implementation

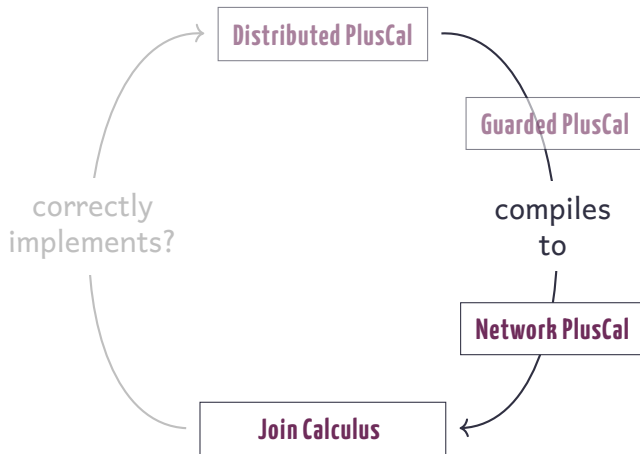












A specification language for distributed/concurrent algorithms...

A specification language for distributed/concurrent algorithms...

- * ...that compiles to a TLA⁺ specification;

A specification language for distributed/concurrent algorithms...

- * ...that compiles to a TLA^+ specification;
- * ...whose syntax is reminiscent of the C programming language;

A specification language for distributed/concurrent algorithms...

- * ...that compiles to a TLA^+ specification;
- * ...whose syntax is reminiscent of the C programming language;
- * ...featuring built-in message-passing and threading constructs.

```
algorithm PingPong {  
  fifos ping, pong;  
  
  process (Ping = 0)  
    variable tmp1 = "";  
  {  
    rcv1:  receive(ping, tmp1);  
          await tmp1 = "Ping";  
          goto pong;  
    pong:  send(pong, "Pong");  
          goto rcv1;  
  }
```

```
  process (Pong = 1)  
    variable tmp2 = "";  
  {  
    ping:  send(ping, "Ping");  
          goto rcv2;  
    rcv2:  receive(pong, tmp2);  
          await tmp2 = "Pong";  
          goto ping;  
  }  
}
```

```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variable tmp1 = "";
  {
    rcv1:  receive(ping, tmp1);
          await tmp1 = "Ping";
          goto pong;
    pong:  send(pong, "Pong");
          goto rcv1;
  }
}

```

```

process (Pong = 1)
  variable tmp2 = "";
{
  ping:  send(ping, "Ping");
        goto rcv2;
  rcv2:  receive(pong, tmp2);
        await tmp2 = "Pong";
        goto ping;
}
}

```



```

algorithm PingPong {
  fifos ping, pong;

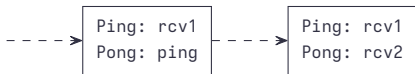
  process (Ping = 0)
    variable tmp1 = "";
  {
    rcv1:  receive(ping, tmp1);
          await tmp1 = "Ping";
          goto pong;
    pong:  send(pong, "Pong");
          goto rcv1;
  }
}

```

```

process (Pong = 1)
  variable tmp2 = "";
{
  ping:  send(ping, "Ping");
        goto rcv2;
  rcv2:  receive(pong, tmp2);
        await tmp2 = "Pong";
        goto ping;
}
}

```




```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variable tmp1 = "";
  {
    rcv1:  receive(ping, tmp1);
          await tmp1 = "Ping";
          goto pong;
    pong: send(pong, "Pong");
          goto rcv1;
  }
}

```

```

process (Pong = 1)
  variable tmp2 = "";
{
  ping: send(ping, "Ping");
        goto rcv2;
  rcv2: receive(pong, tmp2);
        await tmp2 = "Pong";
        goto ping;
}

```



```

algorithm PingPong {
  fifos ping, pong;

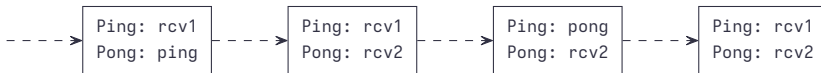
  process (Ping = 0)
    variable tmp1 = "";
  {
    rcv1:  receive(ping, tmp1);
          await tmp1 = "Ping";
          goto pong;
    pong:  send(pong, "Pong");
          goto rcv1;
  }
}

```

```

process (Pong = 1)
  variable tmp2 = "";
{
  ping:  send(ping, "Ping");
        goto rcv2;
  rcv2:  receive(pong, tmp2);
        await tmp2 = "Pong";
        goto ping;
}
}

```



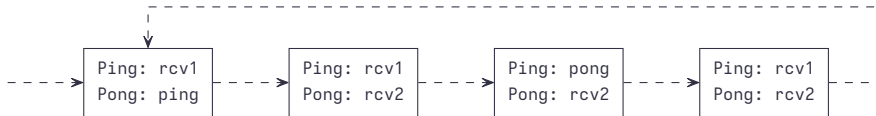
```
algorithm PingPong {
  fifos ping, pong;
```

```
  process (Ping = 0)
    variable tmp1 = "";
```

```
  {
    rcv1:  receive(ping, tmp1);
          await tmp1 = "Ping";
          goto pong;
    pong:  send(pong, "Pong");
          goto rcv1;
  }
```

```
  process (Pong = 1)
    variable tmp2 = "";
```

```
  {
    ping:  send(ping, "Ping");
          goto rcv2;
    rcv2:  receive(pong, tmp2);
          await tmp2 = "Pong";
          goto ping;
  }
```



A distributed process calculus...

A distributed process calculus...

- * ...with compilers to executable code (e.g. JoCaml);

A distributed process calculus...

- * ...with compilers to executable code (e.g. JoCaml);
- * ...based on the metaphor of chemical reactions (atoms, molecules, reactions, ...).

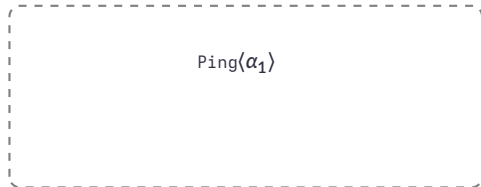
```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>
```

```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>
```

```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<⟩

```

 α_1 

```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<⟩

```

 α_2 


```

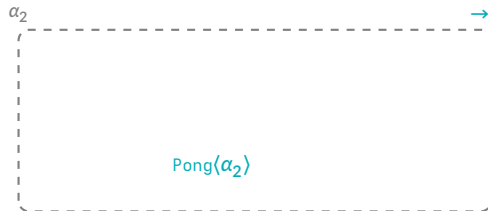
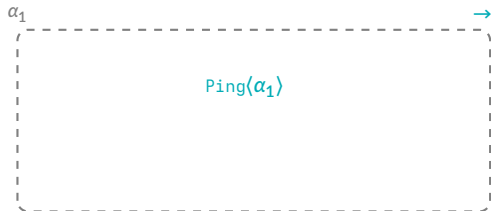
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<⟩

```

```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<⟩

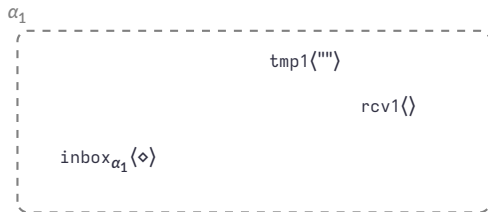
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<⟩

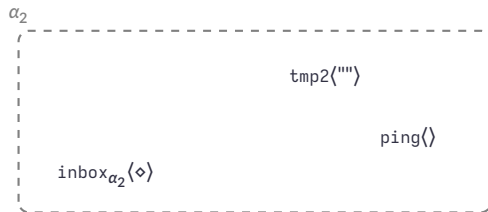
```



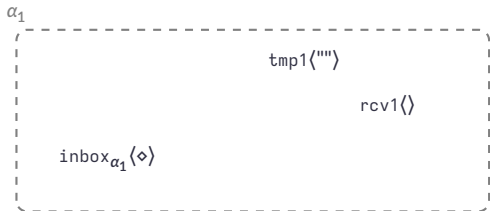
```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<⟩

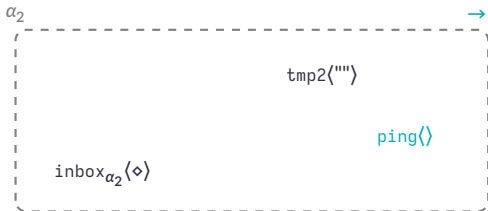
```



```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>
```



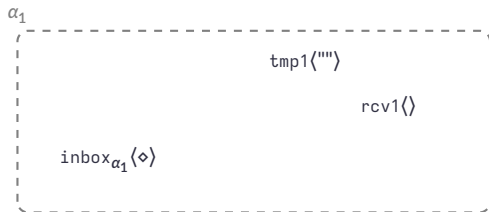
```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≡ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<⟩

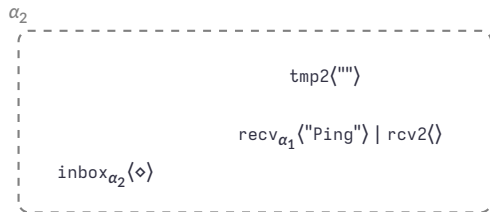
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≡ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<⟩

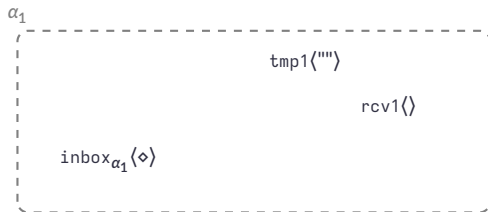
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≡ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

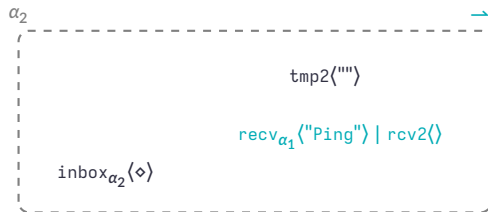
```



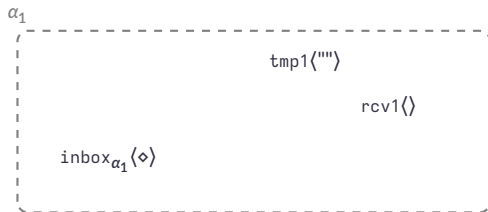
```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≡ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

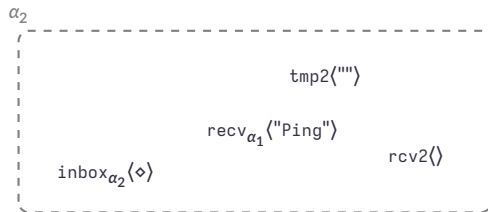
```



```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≝ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>
```

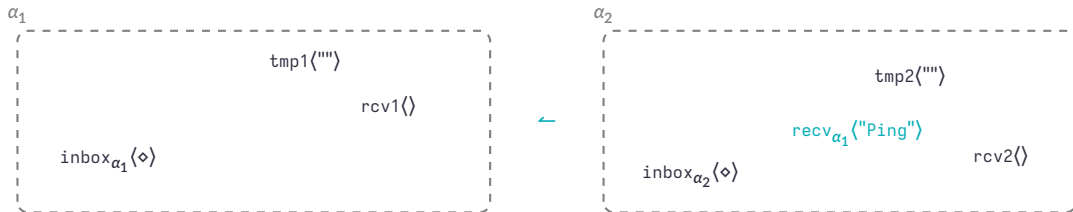


```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≝ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>
```



```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>
```

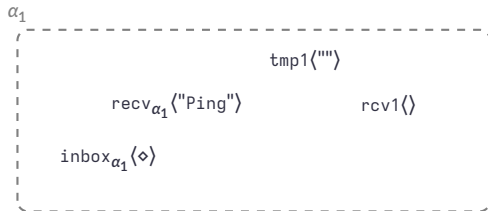
```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

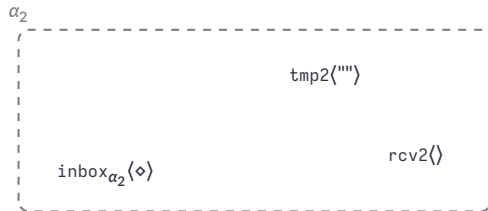
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

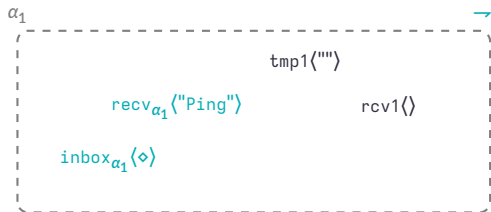
```




```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>

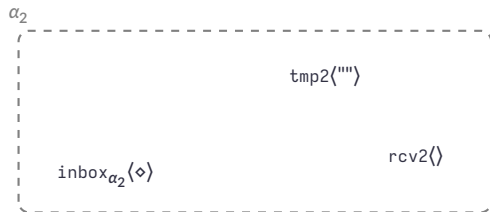
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>

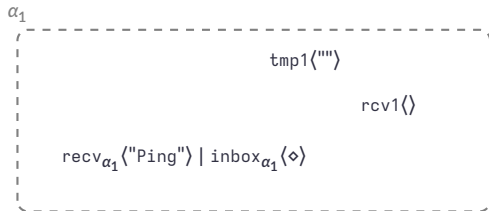
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>

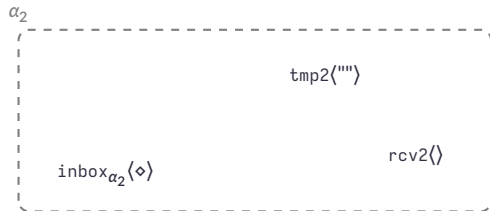
```



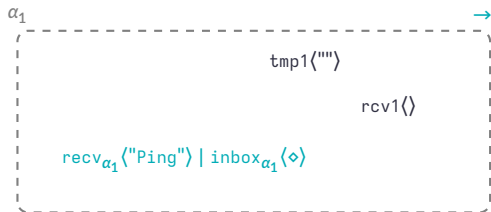
```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>

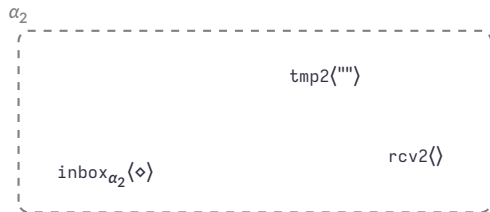
```



```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
  in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>
```



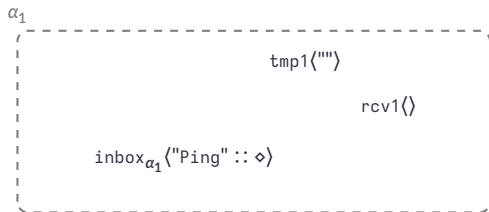
```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
  in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

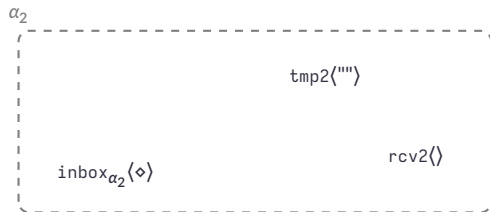
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

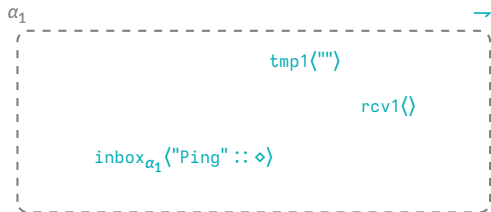
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

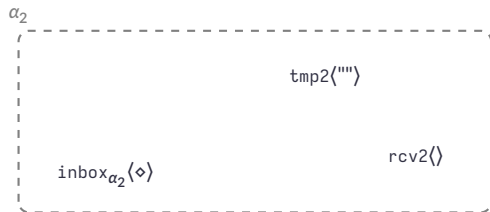
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

```

 α_1

$\text{rcv1}\langle \rangle \mid \text{inbox}_{\alpha_1}\langle \text{"Ping"} :: \diamond \rangle \mid \text{tmp1}\langle \text{""} \rangle$

```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

```

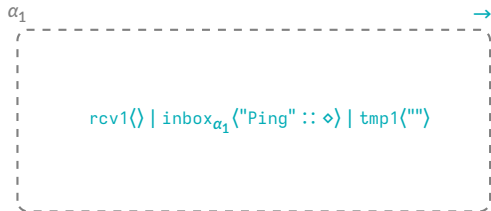
 α_2

$\text{tmp2}\langle \text{""} \rangle$
 $\text{inbox}_{\alpha_2}\langle \diamond \rangle$ $\text{rcv2}\langle \rangle$

```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<>
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>

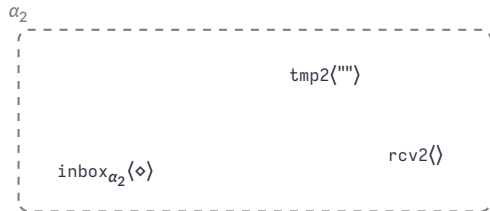
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<> ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<>
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>

```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

```

 α_1

$\text{inbox}_{\alpha_1}\langle\Diamond\rangle \mid \text{tmp1}\langle\text{"Ping"}\rangle \mid \text{pong}\langle\rangle$

```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

```

 α_2

$\text{tmp2}\langle\text{""}\rangle$

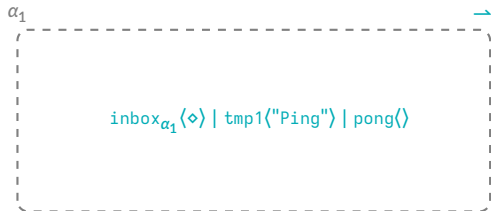
$\text{inbox}_{\alpha_2}\langle\Diamond\rangle$

$\text{rcv2}\langle\rangle$


```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

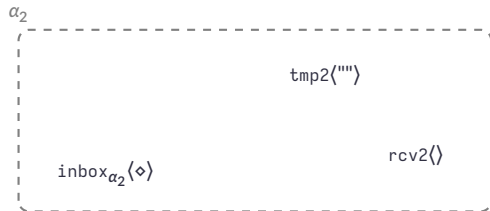
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

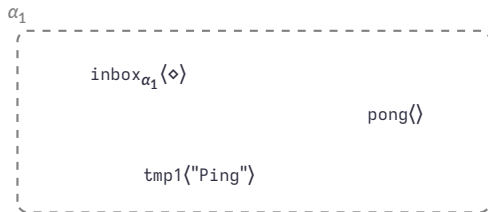
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

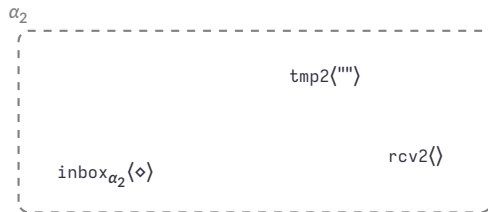
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

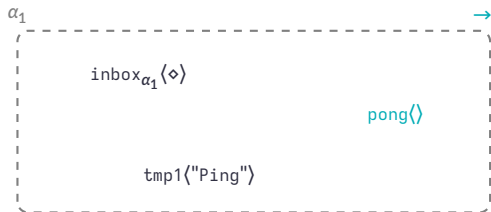
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>

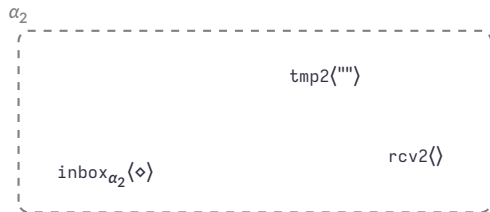
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>

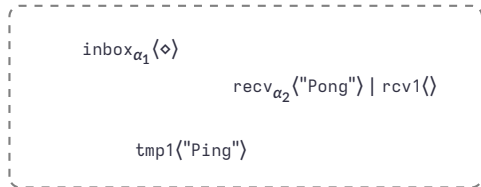
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

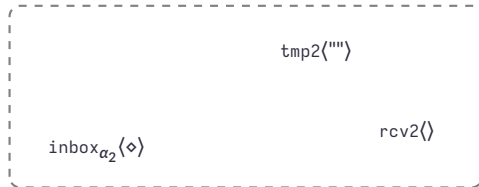
```

 α_1 

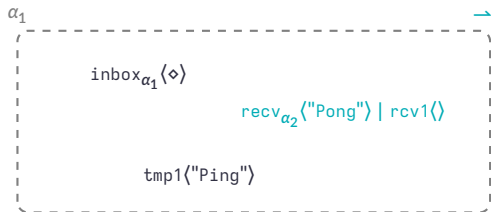
```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

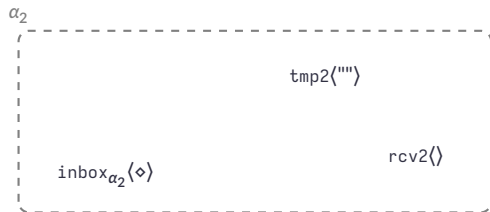
```

 α_2 

```
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
  register recv as "ping";
  inbox<◇> | tmp1<""> | rcv1<>
```



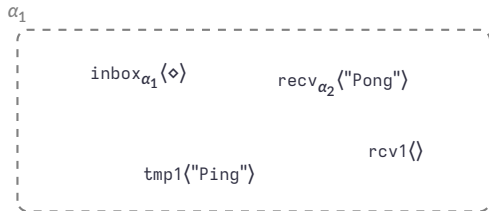
```
def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
  register recv as "pong";
  inbox<◇> | tmp2<""> | ping<>
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

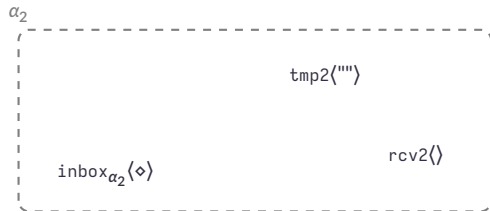
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

```



```

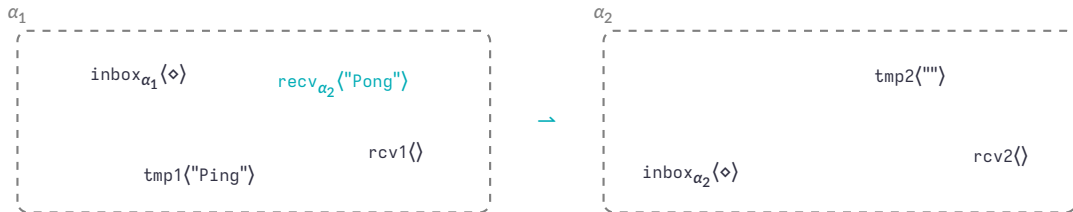
def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or rcv1<> | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox<Tail(vs)> | tmp1<Head(vs)> | pong<>
  or pong<> ▷
    let send ≐ lookup "pong";
    send<"Pong"> | rcv1<>
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<>

```

```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox<Append(vs, v)>
  or ping<> ▷
    let send ≐ lookup "ping";
    send<"Ping"> | rcv2<>
  or rcv2<> | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox<Tail(vs)> | tmp2<Head(vs)> | ping<>
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<>

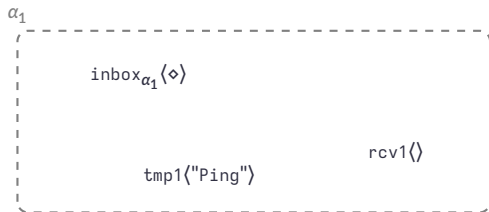
```



```

def Ping<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or rcv1<⟩ | inbox<vs> | tmp1<_>
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      inbox⟨Tail(vs)⟩ | tmp1⟨Head(vs)⟩ | pong<⟩
  or pong<⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1<⟩
in
register recv as "ping";
inbox<◇> | tmp1<""> | rcv1<⟩

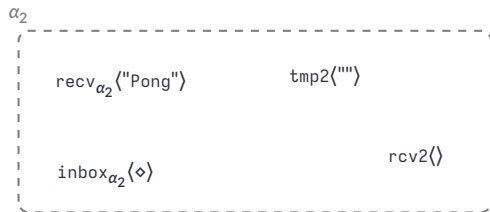
```



```

def Pong<self> ▷
  def recv<v> | inbox<vs> ▷
    inbox⟨Append(vs, v)⟩
  or ping<⟩ ▷
    let send ≐ lookup "ping";
    send⟨"Ping"⟩ | rcv2<⟩
  or rcv2<⟩ | inbox<vs> | tmp2<_>
    if Len(vs) > 0 ∧ Head(vs) = "Pong" ▷
      inbox⟨Tail(vs)⟩ | tmp2⟨Head(vs)⟩ | ping<⟩
in
register recv as "pong";
inbox<◇> | tmp2<""> | ping<⟩

```



⚠ Representation of process-local variables?

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?
⇒ Block atoms appear at most once, each branch consumes it.

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?
⇒ Block atoms appear at most once, each branch consumes it.
- ⚠ Atomicity of PlusCal blocks?

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?
⇒ Block atoms appear at most once, each branch consumes it.
- ⚠ Atomicity of PlusCal blocks?
⇒ Reactions consume atoms on fire, all at once.

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?
⇒ Block atoms appear at most once, each branch consumes it.
- ⚠ Atomicity of PlusCal blocks?
⇒ Reactions consume atoms on fire, all at once.
- ⚠ Inter-process message passing?

- ⚠ Representation of process-local variables?
⇒ Linear *state atoms*.
- ⚠ Handling of non-deterministic choice branches?
⇒ Block atoms appear at most once, each branch consumes it.
- ⚠ Atomicity of PlusCal blocks?
⇒ Reactions consume atoms on fire, all at once.
- ⚠ Inter-process message passing?
⇒ Public atoms used as mailboxes.


```
algorithm PingPong {  
  fifos ping, pong;  
  
  process (Ping = 0)  
    variables  
      tmp1 = "",  
      inbox = ⟨⟩;  
  {  
    rcv1: await Len(inbox) > 0;  
          await Head(inbox) = "Ping";  
          tmp1 := Head(inbox);  
          inbox := Tail(inbox);  
          goto pong;  
    pong: send(pong, "Pong");  
          goto rcv1;  
  }  
  (* @rx: ping → inbox; *) {}  
}
```

```
algorithm PingPong {
  fifos ping, pong;
```

```
  process (Ping = 0)
    variables
      tmp1 = "",
      inbox = ⟨⟩;
  {
    rcv1: await Len(inbox) > 0;
          await Head(inbox) = "Ping";
          tmp1 := Head(inbox);
          inbox := Tail(inbox);
          goto pong;
    pong: send(pong, "Pong");
          goto rcv1;
  }
  (* @rx: ping → inbox; *) {}
}
```

⇒

```
def Ping(self) ▷
```

```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variables
      tmp1 = "",
      inbox = ⟨⟩;
  {
    rcv1: await Len(inbox) > 0;
        await Head(inbox) = "Ping";
        tmp1 := Head(inbox);
        inbox := Tail(inbox);
        goto pong;
    pong: send(pong, "Pong");
        goto rcv1;
  }
  (* @rx: ping → inbox; *) {}
}

```

⇒

```

def Ping(self) ▷
  def recv(v) | inbox(vs) ▷
    inbox⟨Append(vs, v)⟩

  in
  register recv as "ping";

```

```
algorithm PingPong {
  fifos ping, pong;
```

```
  process (Ping = 0)
```

```
    variables
```

```
      tmp1 = "",
```

```
      inbox = ⟨⟩;
```

```
  {
```

```
    rcv1: await Len(inbox) > 0;
```

```
    await Head(inbox) = "Ping";
```

```
    tmp1 := Head(inbox);
```

```
    inbox := Tail(inbox);
```

```
    goto pong;
```

```
  pong: send(pong, "Pong");
```

```
    goto rcv1;
```

```
  }
```

```
  (* @rx: ping → inbox; *) {}
```

```
}
```

⇒

```
def Ping(self) ▷
```

```
  def recv(v) | inbox(vs) ▷
```

```
    inbox⟨Append(vs, v)⟩
```

```
in
```

```
  register recv as "ping";
```

```
  tmp1⟨""⟩ | inbox⟨◇⟩
```

```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variables
      tmp1 = "",
      inbox = ⟨⟩;
  {
    rcv1: await Len(inbox) > 0;
        await Head(inbox) = "Ping";
        tmp1 := Head(inbox);
        inbox := Tail(inbox);
        goto pong;
    pong: send(pong, "Pong");
        goto rcv1;
  }
  (* @rx: ping → inbox; *) {}
}

```

⇒

```

def Ping(self) ▷
  def rcv⟨v⟩ | inbox⟨vs⟩ ▷
    inbox⟨Append(vs, v)⟩

  in
    register rcv as "ping";
    tmp1⟨""⟩ | inbox⟨⟨⟩⟩ | rcv1⟨⟩

```

```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variables
      tmp1 = "",
      inbox = ⟨⟩;
  {
    rcv1: await Len(inbox) > 0;
        await Head(inbox) = "Ping";
        tmp1 := Head(inbox);
        inbox := Tail(inbox);
        goto pong;
    pong: send(pong, "Pong");
        goto rcv1;
  }
  (* @rx: ping → inbox; *) {}
}

```

⇒

```

def Ping(self) ▷
  def rcv⟨v⟩ | inbox⟨vs⟩ ▷
    inbox⟨Append(vs, v)⟩
  or rcv1⟨⟩ | inbox⟨vs⟩ | tmp1⟨_⟩
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      tmp1⟨Head(vs)⟩ | inbox⟨Tail(vs)⟩ | pong⟨⟩

  in
    register rcv as "ping";
    tmp1⟨""⟩ | inbox⟨◇⟩ | rcv1⟨⟩

```

```

algorithm PingPong {
  fifos ping, pong;

  process (Ping = 0)
    variables
      tmp1 = "",
      inbox = ⟨⟩;
  {
    rcv1: await Len(inbox) > 0;
        await Head(inbox) = "Ping";
        tmp1 := Head(inbox);
        inbox := Tail(inbox);
        goto pong;
    pong: send(pong, "Pong");
        goto rcv1;
  }
  (* @rx: ping → inbox; *) {}
}

```

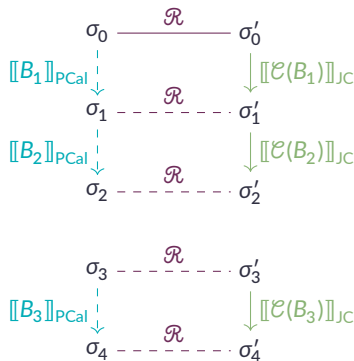
⇒

```

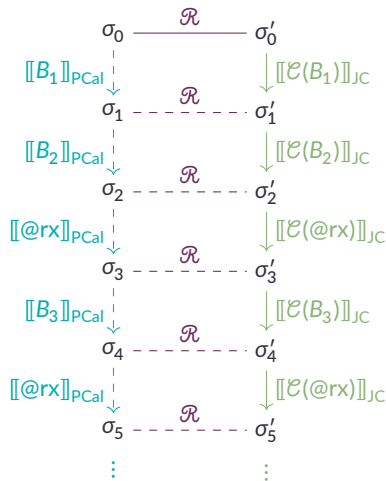
def Ping(self) ▷
  def rcv1(v) | inbox(vs) ▷
    inbox⟨Append(vs, v)⟩
  or rcv1⟨⟩ | inbox(vs) | tmp1⟨_⟩
    if Len(vs) > 0 ∧ Head(vs) = "Ping" ▷
      tmp1⟨Head(vs)⟩ | inbox⟨Tail(vs)⟩ | pong⟨⟩
  or pong⟨⟩ ▷
    let send ≐ lookup "pong";
    send⟨"Pong"⟩ | rcv1⟨⟩
  in
  register rcv1 as "ping";
  tmp1⟨""⟩ | inbox⟨◇⟩ | rcv1⟨⟩

```

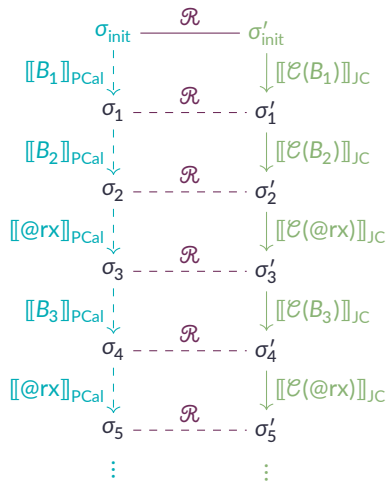

- * Every PlusCal atomic block is "equireducible" with its generated set of reactions.



- * Every PlusCal atomic block is "equireducible" with its generated set of reactions.
- * The reception thread is trivially refined by the reaction on `recv⟨⟩`.



- * Every PlusCal atomic block is "equireducible" with its generated set of reactions.
- * The reception thread is trivially refined by the reaction on `recv()`.
- * Variables and state atoms are initialized to the same values.



A compilation strategy for an executable backend for PlusCal...

A compilation strategy for an executable backend for PlusCal...

- * ...to be instrumentalized;

A compilation strategy for an executable backend for PlusCal...

- * ...to be instrumentalized;
- * ...though not efficiently supported by current implementations of the Join Calculus;

A compilation strategy for an executable backend for PlusCal...

- * ...to be instrumentalized;
- * ...though not efficiently supported by current implementations of the Join Calculus;
- * ...and to be formally proven correct (e.g. in a proof assistant).